endava

endava

Big O and Recursion

MIGUEL ANGEL VILLANUEVA SANCLEMENTE

Agenda

01 Introduction Pag. 4

02 Big O Notation Pag. 7

03 Recursion Pag. 19

01 Introduction

Introduction

Algorithmic Complexity

- The algorithmic complexity is a concept that indicates how much computing resources will be used by an algorithm.
- Usually there are many solutions (algorithms) for a single problem.
- The algorithmic complexity allows us to compare these solutions.
- In Software Development we can compare algorithms using the execution time and memory used.

Introduction

Asymptotic Notations

- The execution time and memory are highly dependent of the machine where the algorithms is executed.
- Asymptotic notations allows to describe mathematically the efficiency of an algorithm.
- Asymptotic notations describes how the time/space of an algorithm grows based on its input. (It doesn't care of where it is executed).
- There 3 types of notations:
  - O Notation: Defines an upper bound on order of growth of time/space.
  - Ω Notation: Defines a lower bound on order of growth of time/space.
  - Θ Notation: Defines the exact order of growth of time/space.

02 Big O Notation

Big O Notation

O Notation

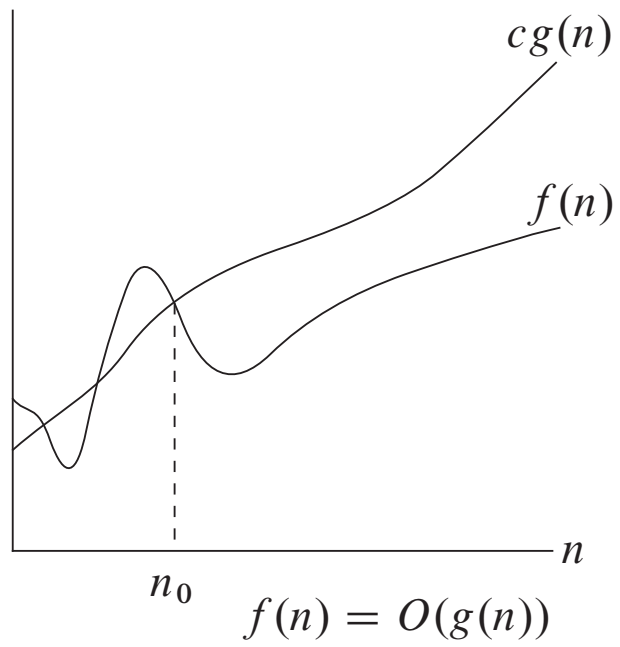
- Is the asymptotic notation most used in software development.
- The growth of an algorithm is represented by a mathematical function.
- It describes how an algorithm would behave in the worst case.

Given two functions $f(n)$ and $g(n)$ we can say that:

$$f(n) \text{ is } O(g(n))$$

If there exists positive constants c and n_0 such than:

$$0 \leq f(n) \leq cg(n), \forall n > n_0$$



Big O Notation

Classification

The complexity of the algorithms could be classify in the follow

O notation	Name	Description
$O(1)$	Constant	Great, easy, fastest, trivial, simple
$O(log\, n)$	Logarithmic	Non-trivial but very manageable
$O(n)$	Linear	Still manageable and predictable, but not optimal for large amounts of data
$O(n \log n)$	Linearithmic	Not quite as good as linear, but not so bad
$O(n^b)$	Polynomial	Now it's getting somewhat expensive
$O(b^n)$	Exponential	Try to avoid this. This is bad, except for very small amounts of data
$O(n!)$	Factorial	Only if you really need it and don't need real-time performance. Or your input is very small

Big O Notation

Example

```
1  insertion_sort(L) →
2      for j ← 2  to length(L) do
3          key ← L[j]
4          i ← j − 1
5          while i > 0 and L[i] > key do
6              L[i + 1] ← L[i]
7              i ← i − 1
8          L[i + 1] ← key
```

Determine how many times the first 4 lines is not complicated.

But how many times the line 5 is executed?

- Suppose $length(L)$ is equal to n
- Line 1 is executed once (1)
- Line 2 is executed n times
- Line 3 is executed $n - 1$ times
- Line 4 is executed $n - 1$ times
- Line 5 ???

Big O Notation

Example

```
1  insertion_sort(L) →
2      for j ← 2  to length(L) do
3          key ← L[j]
4          i ← j − 1
5          while i > 0 and L[i] > key do
6              L[i + 1] ← L[i]
7              i ← i − 1
8          L[i + 1] ← key
```

For each value of j it is executed t_j times.

So we can said that the line 5 is executed:

$$t_2 + t_3 + \cdots + t_n$$

Simplifying:

$$\sum_{j=2}^{n} t_j$$

- Suppose $length(L)$ is equal to n
- Line 1 is executed once (1)
- Line 2 is executed n times
- Line 3 is executed $n - 1$ times
- Line 4 is executed $n - 1$ times
- Line 5 is executed $\sum_{j=2}^{n} t_j$ times

Big O Notation

Example

```
1  insertion_sort(L) →
2      for j ← 2  to length(L) do
3          key ← L[j]
4          i ← j − 1
5          while i > 0 and L[i] > key do
6              L[i + 1] ← L[i]
7              i ← i − 1
8          L[i + 1] ← key
```

The lines 6 and 7 are execute one time less than line 5 so:

$$(t_2 - 1) + (t_3 - 1) + \cdots + (t_n - 1)$$

Simplifying:

$$\sum_{j=2}^{n} (t_j - 1)$$

- Suppose $length(L)$ is equal to n
- Line 1 is executed once (1)
- Line 2 is executed n times
- Line 3 is executed $n - 1$ times
- Line 4 is executed $n - 1$ times
- Line 5 is executed $\sum_{j=2}^{n} t_j$ times
- Line 6 is executed $\sum_{j=2}^{n} (t_j - 1)$ times
- Line 7 is executed $\sum_{j=2}^{n} (t_j - 1)$times
- Line 8 is executed $n - 1$ times

Big O Notation

Example

```
1  insertion_sort(L) →
2      for j ← 2  to length(L) do
3          key ← L[j]
4          i ← j − 1
5          while i > 0 and L[i] > key do
6              L[i + 1] ← L[i]
7              i ← i − 1
8          L[i + 1] ← key
```

We can assume that in the worst case the line 5 always will be executed j times so:

$$\sum_{j=2}^{n} t_j \equiv \sum_{j=2}^{n} j$$

- Suppose $length(L)$ is equal to n
- Line 1 is executed once (1)
- Line 2 is executed n times
- Line 3 is executed $n - 1$ times
- Line 4 is executed $n - 1$ times
- Line 5 is executed $\sum_{j=2}^{n} j$ times
- Line 6 is executed $\sum_{j=2}^{n} (j - 1)$ times
- Line 7 is executed $\sum_{j=2}^{n} (j - 1)$times
- Line 8 is executed $n - 1$ times

Big O Notation

Example

```
1  insertion_sort(L) →
2      for j ← 2  to length(L) do
3          key ← L[j]
4          i ← j − 1
5          while i > 0 and L[i] > key do
6              L[i + 1] ← L[i]
7              i ← i − 1
8          L[i + 1] ← key
```

We need to express the lines 5, 6 and 7 in terms of n.

So we can resolve the summation in line 5:

$$\sum_{j=2}^{n} j = \frac{n(n-1)}{2} - 1$$

$$\sum_{j=2}^{n} (j-1) = \sum_{j=2}^{n} j - \sum_{j=2}^{n} 1 = \frac{n(n-1)}{2} - 1 - n - 1$$

- Suppose $length(L)$ is equal to n
- Line 1 is executed once (1)
- Line 2 is executed n times
- Line 3 is executed $n - 1$ times
- Line 4 is executed $n - 1$ times
- Line 5 is executed $\left(\frac{n(n-1)}{2} - 1\right)$times
- Line 6 is executed $\left(\frac{n(n-1)}{2} - n - 2\right)$ times
- Line 7 is executed $\left(\frac{n(n-1)}{2} - n - 2\right)$times
- Line 8 is executed $n - 1$ times

Big O Notation

Example

We calculate the function that represents the algorithm and reduce it.

$$T(n) = 1 + n + 3(n - 1) + \left(\frac{n(n-1)}{2} - 1\right) + 2\left(\frac{n(n-1)}{2} - n - 2\right)$$

$$T(n) = 1 + n + 3n - 3 + \left(\frac{n^2 - n}{2} - 1\right) + (n^2 - n - 2n - 4)$$

$$T(n) = 4n - 2 + \frac{n^2 - n}{2} - 1 + n^2 - n - 2n - 4$$

$$T(n) = n - 7 + \frac{n^2}{2} - \frac{n}{2} + n^2$$

$$T(n) = \frac{3n^2}{2} - \frac{n}{2} - 7$$

- Suppose $length(L)$ is equal to n
- Line 1 is executed once (1)
- Line 2 is executed n times
- Line 3 is executed $n - 1$ times
- Line 4 is executed $n - 1$ times
- Line 5 is executed $\left(\frac{n(n-1)}{2} - 1\right)$times
- Line 6 is executed $\left(\frac{n(n-1)}{2} - n - 2\right)$ times
- Line 7 is executed $\left(\frac{n(n-1)}{2} - n - 2\right)$times
- Line 8 is executed $n - 1$ times

Big O Notation

Example

Finally, the function $T(n)$ should be classified.

$$T(n) = \frac{3n^2}{2} - \frac{n}{2} - 7$$

As $T(n)$ is a function of grade 2. We can assume that it is $O(n^2)$.

$$f(n) = T(n)$$
$$g(n) = n^2$$
$$0 \leq f(n) \leq cg(n)$$
$$0 \leq \frac{3n^2}{2} - \frac{n}{2} - 7 \leq cn^2$$

Multiply by 2

$$0 \leq 3n^2 - n - 14 \leq 2cn^2$$

Divide by n^2

$$0 \leq 3 - \frac{1}{n} - \frac{14}{n^2} \leq 2c$$

When n is too big we can say that $\frac{x}{n}$ is 0 when x is a constant.

$$0 \leq 3 \leq 2c$$
$$c \geq \frac{3}{2}$$

Now we need to find the value of n_o

$$0 \leq \frac{3n^2}{2} - \frac{n}{2} - 7 \leq \frac{3n^2}{2}$$
$$0 \leq 3n^2 - n - 14 \leq 3n^2$$

The inequality is only true if

$$3n^2 \geq n + 14$$
$$3n^2 - n - 14 = 0$$

Solving quadratic equation

$$n \approx 2,\overline{33}$$

n should be a positive integer so:

$$n_o = 3, c = \frac{3}{2}$$

Big O Notation

Other Asymptotic Notations

- There are other 2 types of asymptotic notations: Ω Notation and Θ Notation.
- These notations are similar than O Notation.

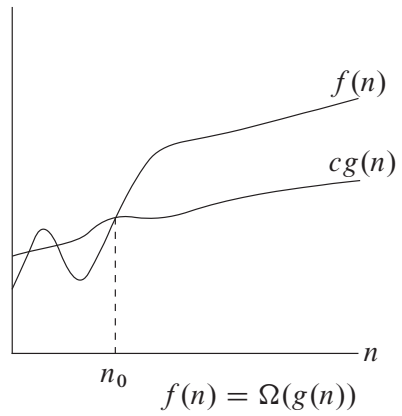
Ω Notation

Given two functions $f(n)$ and $g(n)$ we can say that:

$f(n)$ is $O(g(n))$

If there exists positive constants c and n_0 such than:

$$0 \leq cg(n) < f(n), \forall n > n_0$$



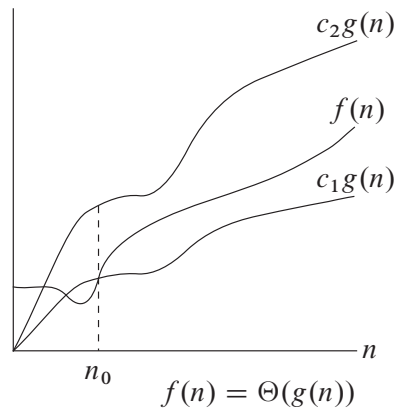
Θ Notation

Given two functions $f(n)$ and $g(n)$ we can say that:

$f(n)$ is $O(g(n))$

If there exists positive constants c_1, c_2 and n_0 such than:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n > n_0$$



Big O Notation

Small Notations

- There is a special case when we are sure than the function $f(n)$ won't be equal to the asymptotic function.
- This case are known as little notation.

o Notation

Given two functions $f(n)$ and $g(n)$ we can say that:

$f(n)$ is $O(g(n))$

If there exists positive constants c and n_0 such than:

$$0 \leq f(n) < cg(n), \forall n > n_0$$

ω Notation

Given two functions $f(n)$ and $g(n)$ we can say that:

$f(n)$ is $O(g(n))$

If there exists positive constants c and n_0 such than:

$$0 \leq cg(n) < f(n), \forall n > n_0$$

03 Recursion

Recursion

What is recursion?

- Occurs when the definition of a concept or process depends on a simpler or previous version of itself.
- A recursive function is a function defined in terms of itself.
- Recursion allows us to write smaller, more concise algorithms.
- The recursive function has a stop condition and a recursive part.

$$f(a,b) = \begin{cases} a, & b = 0 \\ f(a+1, b-1) & b > 0 \end{cases}$$

$$\begin{aligned} & f(a,b) \rightarrow \\ & \quad if\ b = 0\ then \\ & \qquad return\ a \\ & \quad else \\ & \qquad return\ f(a+1, b-1) \end{aligned}$$

Recursion

Classic examples

Factorial

$$fact(n) = \begin{cases} 1, & n = 0 \\ n * fact(n-1) & n > 0 \end{cases}$$

```
fact(n) →
    if n = 0 them
        return 1
    else
        return n * fact(n − 1)
```

Fibonacci

$$fibo(n) = \begin{cases} 1, & n = 0 \vee n = 1 \\ fibo(n-1) + fibo(n-2) & n > 1 \end{cases}$$

```
fibo(n) →
    if n = 0 or n = 1 them
        return 1
    else
        return fibo(n − 1) + fibo(n − 2)
```

Recursion

Complexity of a Recursive Algorithm

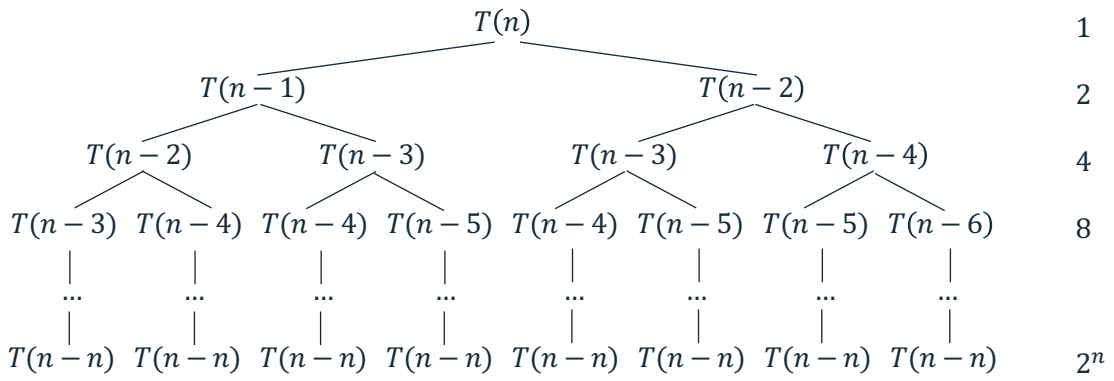
- Obtain a function that represents how a recursive algorithm grows is not than simple.
- There are 3 methods that can be used to obtain the complexity of a recursive algorithm
  - Sustitution method
  - Iteration method or Recursion Tree method
  - Master method

Recursion

Fibonacci

Using the iteration method we can get the complexity of fibonacci.

$$T(n) = T(n-1) + T(n-2)$$



In the worst case we can say that $T(n)$ is $O(2^n)$

Recursion

Iteration

- An iteration during its execution use and additional constant space, doesn't matter the its duration.
- An iteration is characterized by starting with an initial state S_0 and transform that state to a set of intermediate states until reach a final state S_f.

$$S_o \rightarrow S_1 \rightarrow S_2 \rightarrow \cdots \rightarrow S_f$$

- An iterative algorithm has the follow schema:

$$\begin{aligned} & S \leftarrow S_0 \\ & \textit{while is not } final(S) \textit{ do} \\ & \qquad S \leftarrow transform(S) \end{aligned}$$

- The iteration are usually implemented using the control structures: for and while.

Recursion

Iterative Fibonacci

- We can use an iterative strategy to solve the fibonnaci in more efficient way.
- We define the state as:

$$S = \{prev, result, count\}$$

- The initial state would be:

$$S_o = \{1,1,1\}$$

- Transformation of the states:

$$\{p, r, c\} \rightarrow \{r, p + r, c + 1\}$$

- The final state:

$$S_f = \{result_{S_{f-1}}, result, n\}$$

```
fibo(n) →
    prev, result, count ← 1
    while count < n do
        temp ← prev
        prev ← result
        result ← temp + result
        count ← count + 1
    return result
```

```
fibo(n) →
    prev, result ← 1
    for count ← 1 to (n − 1) do
        temp = prev
        prev = result
        result = temp + result
    return result
```

Recursion

Iterative Fibonacci

- We can conclude that the iterative Fibonacci is $O(n)$ since the for and while will be executed n times.
- We can write iterative algorithms using recursion.

$$T(n,p,r,c) = T(n-1,r,p+r,c+1) = T(n-1,p_{n-1},r_{n-1},c_{n-1})$$
$$T(n,p_n,r_n,c_n)$$
$$T(n-1,p_{n-1},r_{n-1},c_{n-1})$$
$$T(n-2,p_{n-2},r_{n-2},c_{n-2})$$
$$T(n-3,p_{n-3},r_{n-3},c_{n-3})$$
$$\dots$$
$$T(1,p_1,r_1,c_1)$$

```
fibo(n) →
  return fibo_aux(n, 1,1,1)

fibo_aux(n, prev, result, count) →
  if count ≥ n them
    return result
  else
    return fibo_aux(n, result, result + prev, count + 1)
```

- $fibo_aux$ is called n times.
- So, we can conclude that $fibo_aux$ is $O(n)$

Thank you!